PL - 2 Adeti C. Jamdhade A batch SOC-8.

Q1. 1) list : Ordered sequence
• Mutable (elements can be changed after creation)

2) Tuple : Ordered sequence
• Immutable (Cannot be changed once created.

3) String : Ordered sequence
• Immutable (characters cannot be modified

4) Range, ordered sequence no.

Q2. Indexing in python sequence is used to access a single element from a sequence Indexing starts from 0 (left to right) negative indexing starts from -1 (right to left)

eg: Code

```
v = "python"
l = [10,20,30,40]

print [s[10]]
print [s[-1])
print (l[1])
print (l[-2])
```

Slicing in python sequence is used to extract a part (subsequence) of a sequence.

Syntax:

Sequence [start: Stop: step]

start → starting index (included)
stop → ending index (excluded)
step → gap between elements (optional)

```
print(s[1:4])
print(s[0:3])
print(s[::2])
print(1[::-1])
```

Q3. Write a python program that uses indexing & slicing on a string or list, and applies at least two built-in function to display working ful result.

python code.

```
text = "python programming"
# indexing
first_char = text[0]
last_char = text[-1]

#slicing
sub_text = text[0:6]
way_second = text[::2]
```

length = len(text)
max_char = max(text)

print("Original string:", text)
print("First character:", first_char)
print("Last character:", last_char)

print("Sliced string (0:6):", sub_text)

print("Every second char:", every_second)
print("Length of string:", length)
print("Maximum char:", max_char)

4. Difference between == and is in python. The equality operates (==) compares the values location of two objects. The identity operate (is) compares the memory location of two objects.
Two sequence may have the same value but be stored in different memory location so == returns while the return is false.
for mutual sequences like list, if two variable refers to the same objects changes made through one variable affect the other.
for immutable sequence like string & tuple, values cannot be modified any changes a new object reducing side effects

Q5. When storing & managing unique student records structure affects efficiency, data integrity & ease of access.

1. list: list stores elements in an ordered sequence They allow duplicate entries. So extra logic is required to ensure uniquely searching for a student record is slow (O) specially for larger datasets.

2. Sets: Set stores only unique elements automatically they are unordered & do not support indexing. Fast membership checking (O(1)) avg(time)

Dictionaries store the data, as key value pairs Keys are unique, ensuring as duplicate student IDs.
Very fast access, insertion & deletion (O(1)) average time.
Can store structured & nested as values.

Q6. Student = {
1 : {"Name" : "Amit" "Marks" : [75, 80, 95]}
2 : {"name" : "Neha" "Marks" : [88, 91, 87]}
}

for val data in student.items():
marks = data("Marks")

```
for roll data in student.items():
    marks = data["Marks"]

first_marks = marks[0]
top_two = sorted(Marks, reverse=True)[:2]

total = sum(Marks)
average = total / len(Marks)

print(f"\n Roll No : {ROll}")
Print(F"name : { data['name']}")
Print(f" Marks : { Marks}")
Print(F" first Mark : { first Mark}")
Print(f "Top two mark : { top_two}")
Print(f " Total : { total} Average : { average :")
```

Q7.

```
text = input("Enter a string : ")
print(" first character : ", text + [0])
print(" last characters : " text [-1])
```

text [0] → access the first character
text [-1] → access the last character using negative Indexing.

8.
```
My_tuple = (1, 2, 3, 4)
My_set = (5, 10, 15)
My_dict = ("name": "Amit",
           "age": 20
           "branch": "comp")
}
print ("list": My_list)
print ("Tuple: My_tuple)
print ("set: " My_set)
print ("Dictionary": My_dict)
```

9.
```
text = "python"
print( "string : " text)
print ("First - char : text[0])
print-("second - char : "text(1])
print("last char : " text [-1])

number. = (10, 20, 30, 40 )
print ("In list" number)
```

10.
```
Numbers = [10, 20, 30, 40, 50]

print (" list : " numbers)

print (" length of list: " len (Numbers))
print ("max value : Max (Numbers))
print ("min value : " Max (Numbers))
print ("sum of element : "Sum (Numbers))
print (" sorted list : Sorted(Numbers).
```